Cyber Attack Prediction: From Traditional Machine Learning to Generative Artificial Intelligence

Cyber-attacks are intended to perform malicious activities such as corrupting user files, stealing data, hacking and many more on victim system. Traditional algorithms unable to detect all types of attacks developed by attackers at different times. To enhance detection accuracy author of this paper giving huge survey on different ML, DL, NLP, Explainable AI and Generative AI algorithms and to main purpose is to identify algorithm who can detect all types of cyber-attack with highest accuracy.

In propose paper author has experimented with many algorithms such as SVM, Random Forest, KNN, XGBOOST, CNN, DNN and many more and each algorithm got trained on many cyber-attack datasets such as NSL-KDD, KDDCUP, UNSW15, CICIDS 2017, CICDDOS 2019, XIOTT and many more. Each algorithm performance is evaluated in terms of accuracy. All algorithms able to detect attacks between an accuracy of 90 to 99% and this experimentation will help researchers in choosing algorithms with highest accuracy.

Propose paper presents state-of-the-art benchmark cyber-attack datasets, ML and DL, and techniques for various cyber-attack predictions. In the domain of cyber-security, transparency, and explainability are essential for combating cyber threats and effectively analysing security decisions. Hence, this study provides a thorough overview of cutting-edge research on XAI for Cybersecurity applications. Propose paper describe the fundamental principles and taxonomies of state-of-the-art XAI models, along with indispensable tools, such as a comprehensive framework and accessible datasets. Propose work will be a valuable resource for researchers, developers, and security professionals seeking to leverage ML, DL, XAI, and GenAI models to address complex challenges within Cybersecurity domains.

Note: for explanation we have used SHAP tool and as Generated AI author has used CHATGPT tool which is not free of cost so we have not implemented this algorithm. Training all datasets with all algorithms may take hours of time so we have implemented important algorithms with different datasets such as NSL KDD, CIC2017, CIC2019 and XIIOT dataset. All 4 datasets trained with different ML and DL algorithms such as DNN, MLP, KNN, SVM, Naïve Bayes, Logistic Regression, and Random Forest.

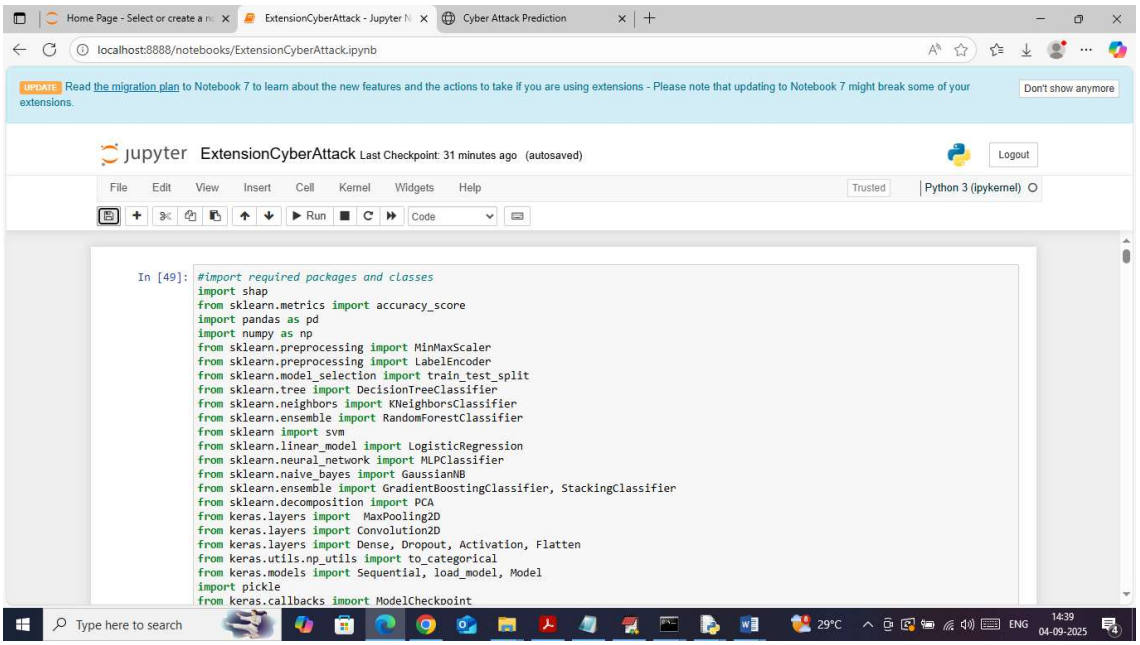
Extension Concept

In paper author has mentioned in future work he will be implemented advance algorithms like Hybrid Model, Adversarial Learning and Multimodal Learning to

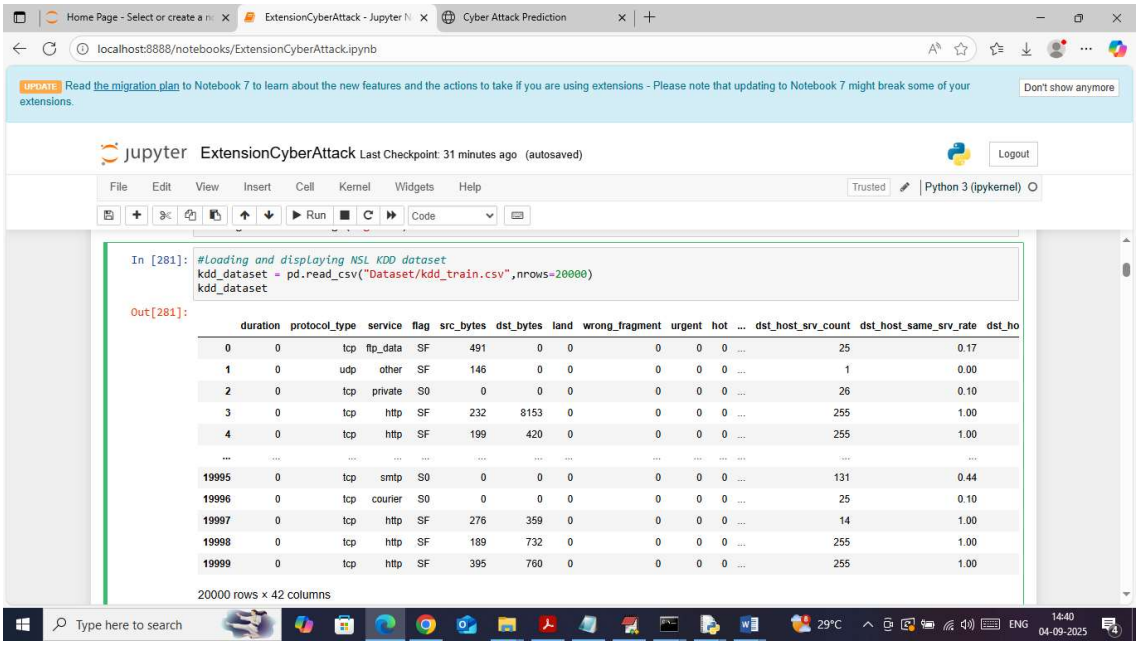
further enhance cyber-attack prediction accuracy. So we have experimented with all the above models and found Hybrid model able to predict attacks from all four dataset with an accuracy between 99 to 100%. As hybrid model we have combined multiple base models like MLP, KNN and Random Forest using Stacked Classifier. Stacked classifier will trained all base models and then choose model with highest accuracy.

SCREEN SHOTS

We have coded this project using JUPYTER notebook and then used separate notebook for propose and extension work. In below screen showing model training, evaluation, dataset processing and visualization code and output with blue colour comments.



In above screen importing required python classes and packages



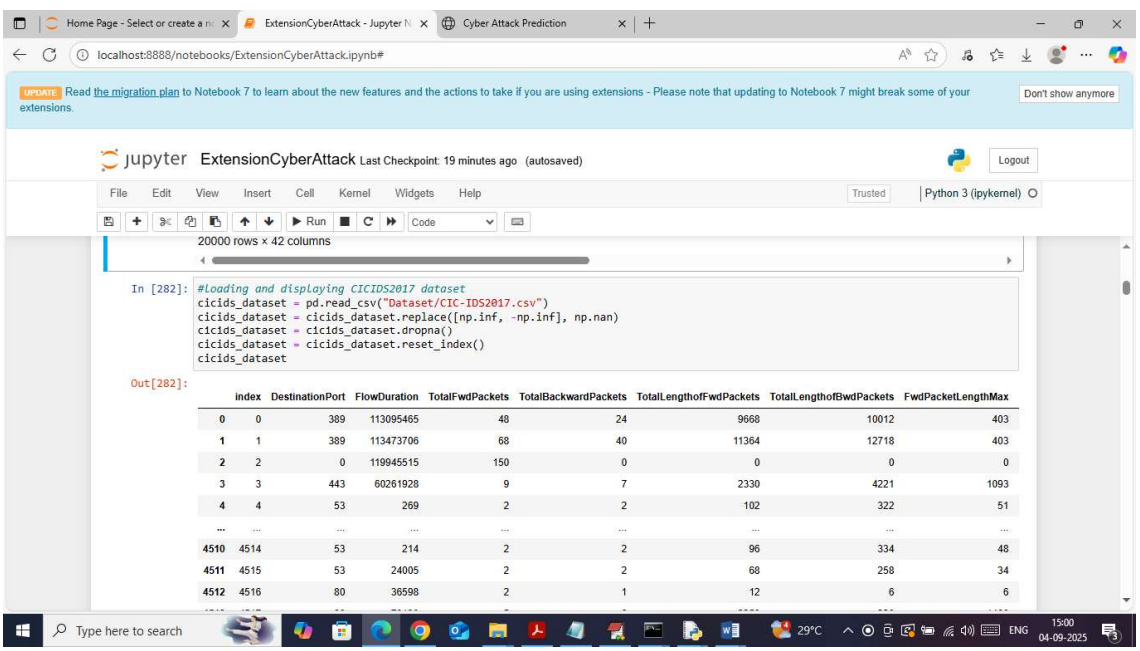
```
In [281]: #Loading and displaying NSL KDD dataset
          kdd_dataset = pd.read_csv("Dataset/kdd_train.csv",nrows=20000)
          kdd_dataset
```

Out[281]:

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_ho
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	25	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	1	0.00	
2	0	tcp	private	S0	0	0	0	0	0	0	...	26	0.10	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	255	1.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	255	1.00	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
19995	0	tcp	smtp	S0	0	0	0	0	0	0	...	131	0.44	
19996	0	tcp	courier	S0	0	0	0	0	0	0	...	25	0.10	
19997	0	tcp	http	SF	276	359	0	0	0	0	...	14	1.00	
19998	0	tcp	http	SF	189	732	0	0	0	0	...	255	1.00	
19999	0	tcp	http	SF	395	760	0	0	0	0	...	255	1.00	

20000 rows × 42 columns

In above screen loading and displaying values of NSL KDD dataset

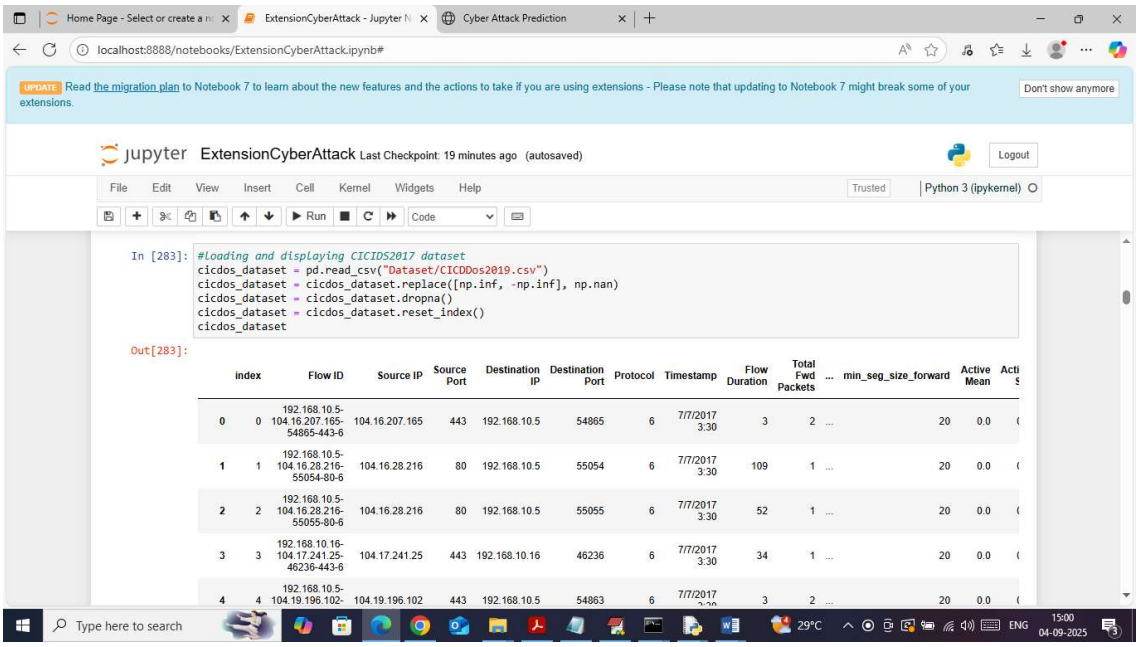


```
In [282]: #Loading and displaying CICIDS2017 dataset
          cicids_dataset = pd.read_csv("Dataset/CIC-IDS2017.csv")
          cicids_dataset = cicids_dataset.replace([np.inf, -np.inf], np.nan)
          cicids_dataset = cicids_dataset.dropna()
          cicids_dataset = cicids_dataset.reset_index()
          cicids_dataset
```

Out[282]:

	index	DestinationPort	FlowDuration	TotalFwdPackets	TotalBackwardPackets	TotalLengthofFwdPackets	TotalLengthofBwdPackets	FwdPacketLengthMax
0	0	389	113095465	48	24	9668	10012	403
1	1	389	113473706	68	40	11364	12718	403
2	2	0	119945515	150	0	0	0	0
3	3	443	60261928	9	7	2330	4221	1093
4	4	53	269	2	2	102	322	51
...	...	...	...	...	...	...	...	...
4510	4514	53	214	2	2	96	334	48
4511	4515	53	24005	2	2	68	258	34
4512	4516	80	36598	2	1	12	6	6

In above screen loading and displaying CICIDS2017 dataset values

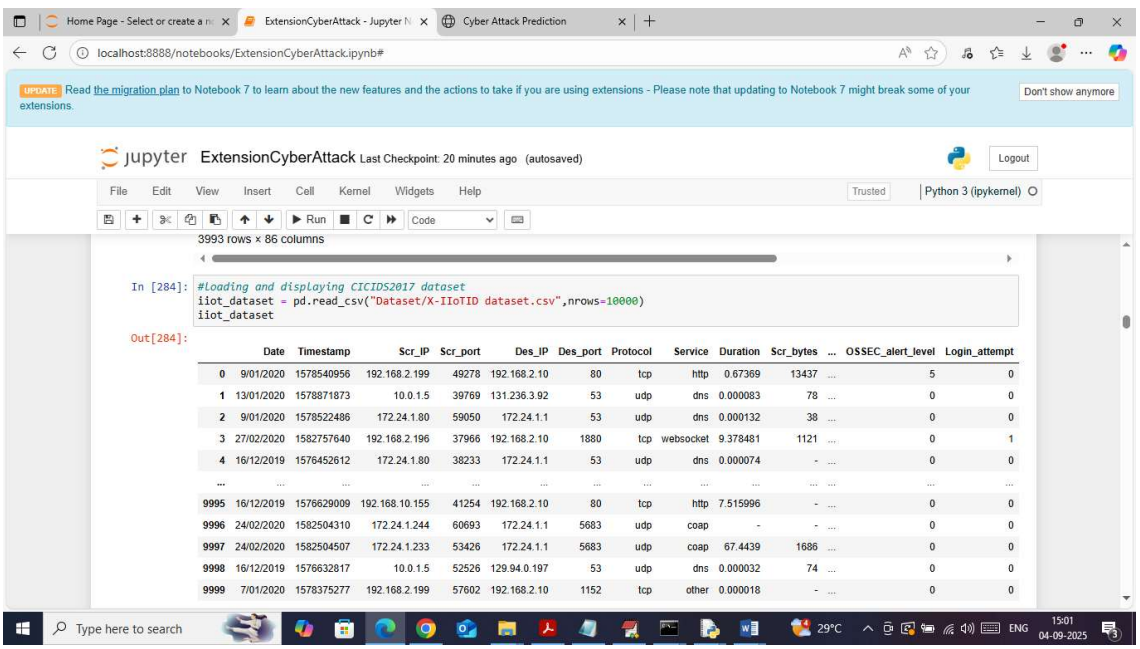


```
In [283]: #Loading and displaying CICIDS2017 dataset
cicdos_dataset = pd.read_csv("Dataset/CICDDos2019.csv")
cicdos_dataset = cicdos_dataset.replace([np.inf, -np.inf], np.nan)
cicdos_dataset = cicdos_dataset.dropna()
cicdos_dataset = cicdos_dataset.reset_index()
cicdos_dataset
```

Out[283]:

	index	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Flow Duration	Total Fwd Packets	...	min_seg_size_forward	Active Mean	Acti S
0	0	192.168.10.5-104.16.207.165-54865-443-6	104.16.207.165	443	192.168.10.5	54865	6	7/7/2017 3:30	3	2	...	20	0.0	(
1	1	192.168.10.5-104.16.28.216-55054-80-6	104.16.28.216	80	192.168.10.5	55054	6	7/7/2017 3:30	109	1	...	20	0.0	(
2	2	192.168.10.5-104.16.28.216-55055-80-6	104.16.28.216	80	192.168.10.5	55055	6	7/7/2017 3:30	52	1	...	20	0.0	(
3	3	192.168.10.16-104.17.241.25-46236-443-6	104.17.241.25	443	192.168.10.16	46236	6	7/7/2017 3:30	34	1	...	20	0.0	(
4	4	192.168.10.5-104.19.196.102-	104.19.196.102	443	192.168.10.5	54863	6	7/7/2017	3	2	...	20	0.0	(

In above screen loading and displaying CICDDOS2019 dataset values



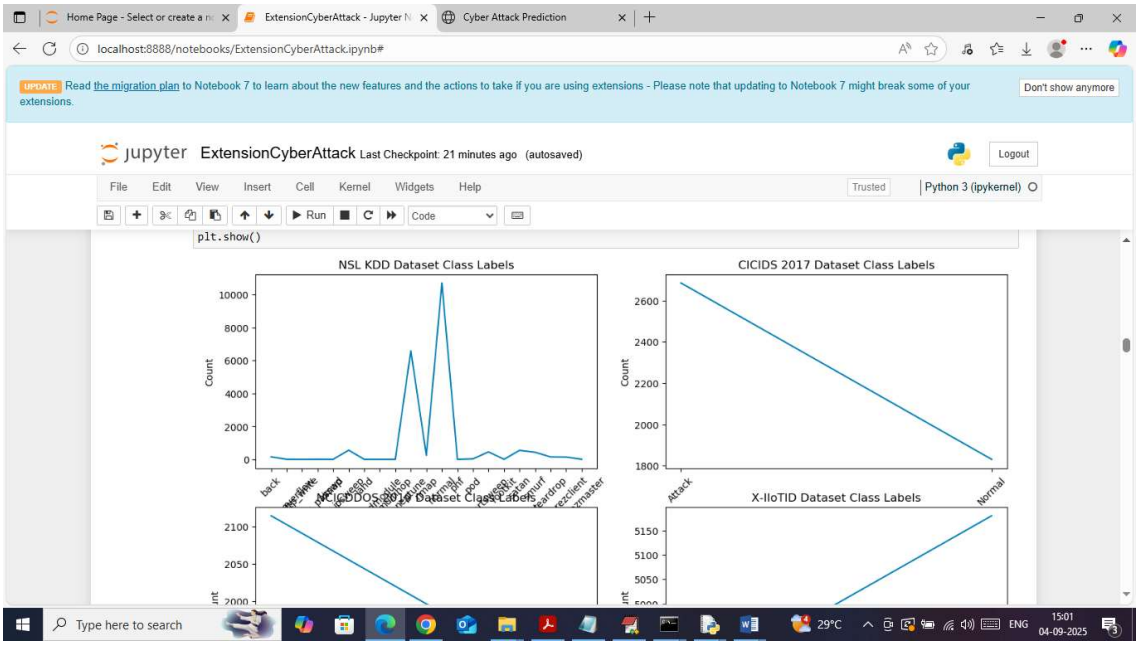
3993 rows × 86 columns

```
In [284]: #Loading and displaying CICIDS2017 dataset
iiot_dataset = pd.read_csv("Dataset/X-IIoTID dataset.csv",nrows=10000)
iiot_dataset
```

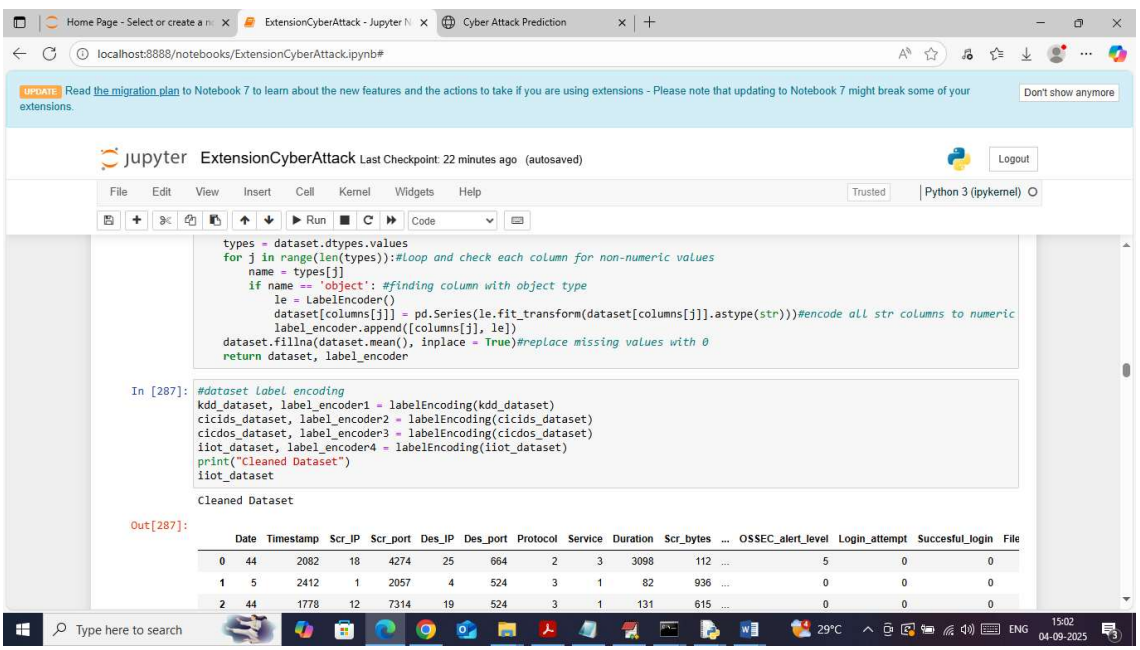
Out[284]:

	Date	Timestamp	Scr_IP	Scr_port	Des_IP	Des_port	Protocol	Service	Duration	Scr_bytes	...	OSSEC_alert_level	Login_attempt
0	9/01/2020	1578540956	192.168.2.199	49278	192.168.2.10	80	tcp	http	0.67369	13437	...	5	0
1	13/01/2020	1578871873	10.0.1.5	39769	131.236.3.92	53	udp	dns	0.000083	78	...	0	0
2	9/01/2020	1578522486	172.24.1.80	59050	172.24.1.1	53	udp	dns	0.000132	38	...	0	0
3	27/02/2020	1582757640	192.168.2.196	37966	192.168.2.10	1880	tcp	websocket	9.378481	1121	...	0	1
4	16/12/2019	1576452612	172.24.1.80	38233	172.24.1.1	53	udp	dns	0.000074	-	...	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	16/12/2019	1576629009	192.168.10.155	41254	192.168.2.10	80	tcp	http	7.515996	-	...	0	0
9996	24/02/2020	1582504310	172.24.1.244	60693	172.24.1.1	5683	udp	coap	-	-	...	0	0
9997	24/02/2020	1582504507	172.24.1.233	53426	172.24.1.1	5683	udp	coap	67.4439	1686	...	0	0
9998	16/12/2019	1576632817	10.0.1.5	52526	129.94.0.197	53	udp	dns	0.000032	74	...	0	0
9999	7/01/2020	1578375277	192.168.2.199	57602	192.168.2.10	1152	tcp	other	0.000018	-	...	0	0

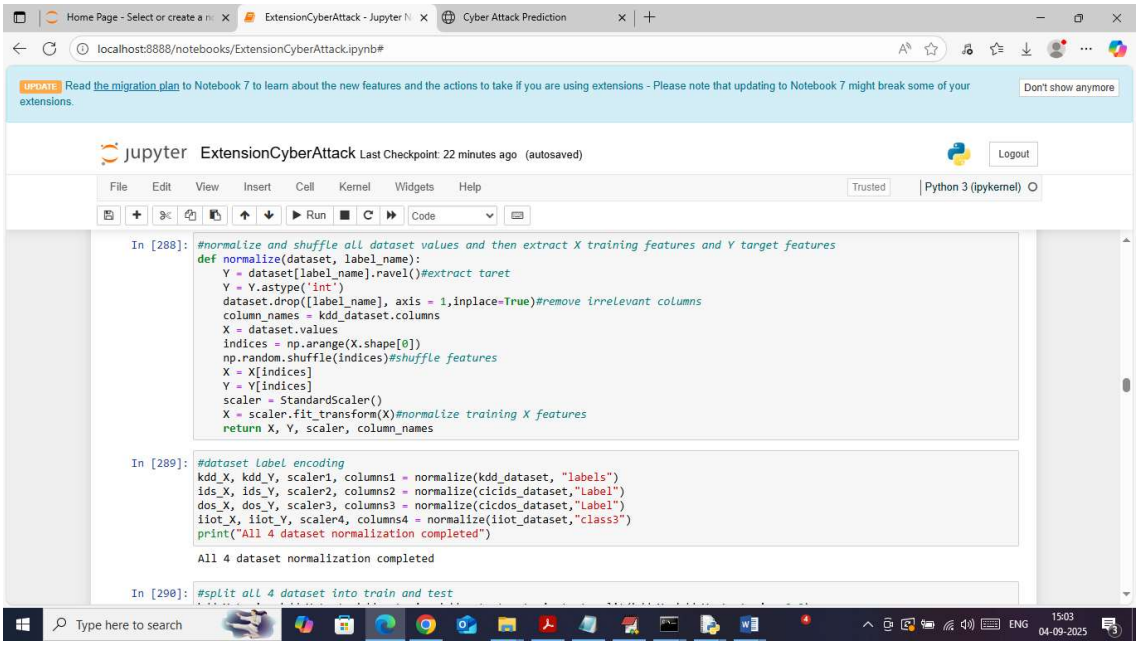
In above screen loading and displaying XIOTIOT dataset values



In above screen visualizing graph of different attacks found in dataset where x-axis represents attack names from each dataset and y-axis represents counts



In above screen applying label encoding on 4 datasets to convert non-numeric data to numeric data and then displaying clean data values



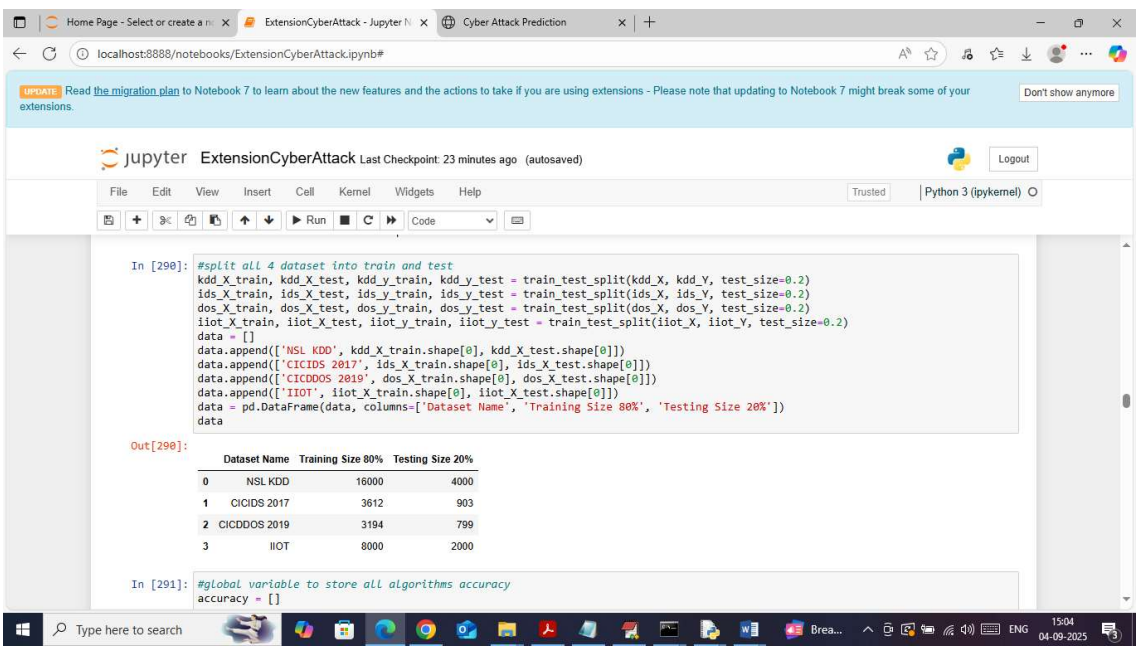
```
In [288]: #normalize and shuffle all dataset values and then extract X training features and Y target features
def normalize(dataset, label_name):
    Y = dataset[label_name].ravel()#extract taret
    Y = Y.astype('int')
    dataset.drop([label_name], axis = 1,inplace=True)#remove irrelevant columns
    column_names = kdd_dataset.columns
    X = dataset.values
    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)#shuffle features
    X = X[indices]
    Y = Y[indices]
    scaler = StandardScaler()
    X = scaler.fit_transform(X)#normalize training X features
    return X, Y, scaler, column_names

In [289]: #dataset label encoding
kdd_X, kdd_Y, scaler1, columns1 = normalize(kdd_dataset, "labels")
ids_X, ids_Y, scaler2, columns2 = normalize(cicids_dataset,"Label")
dos_X, dos_Y, scaler3, columns3 = normalize(cicdos_dataset,"Label")
iiot_X, iiot_Y, scaler4, columns4 = normalize(iiot_dataset,"class3")
print("All 4 dataset normalization completed")

All 4 dataset normalization completed

In [290]: #split all 4 dataset into train and test
```

In above screen defining function to normalize and shuffle values from all dataset values



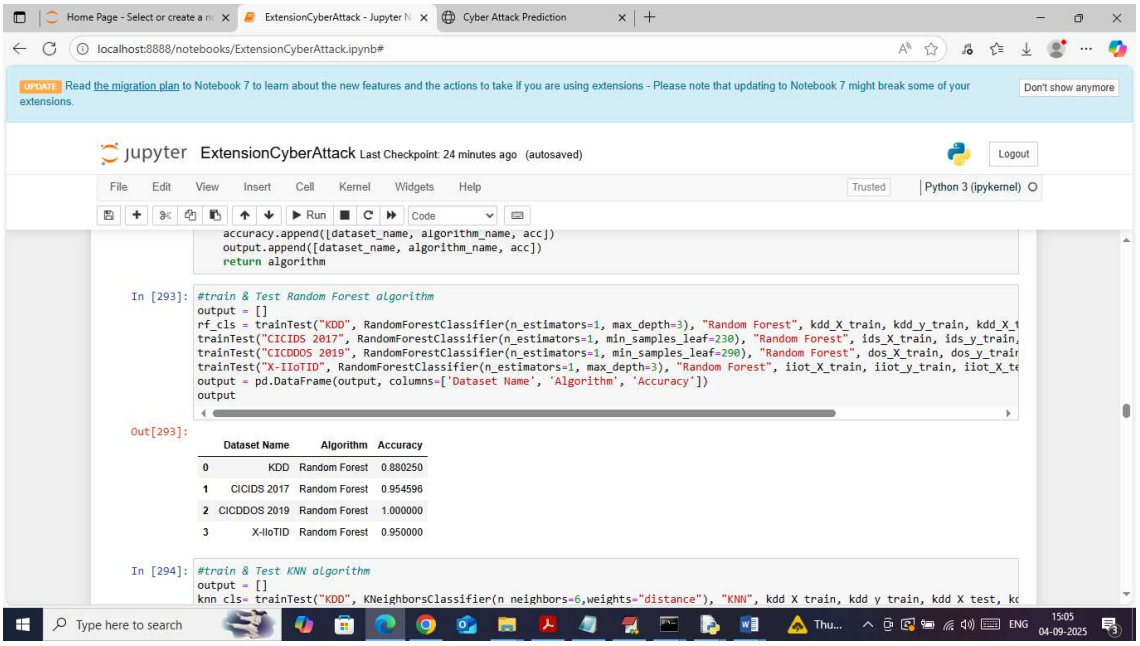
```
In [290]: #split all 4 dataset into train and test
kdd_X_train, kdd_X_test, kdd_y_train, kdd_y_test = train_test_split(kdd_X, kdd_Y, test_size=0.2)
ids_X_train, ids_X_test, ids_y_train, ids_y_test = train_test_split(ids_X, ids_Y, test_size=0.2)
dos_X_train, dos_X_test, dos_y_train, dos_y_test = train_test_split(dos_X, dos_Y, test_size=0.2)
iiot_X_train, iiot_X_test, iiot_y_train, iiot_y_test = train_test_split(iiot_X, iiot_Y, test_size=0.2)
data = []
data.append(['NSL KDD', kdd_X_train.shape[0], kdd_X_test.shape[0]])
data.append(['CICIDS 2017', ids_X_train.shape[0], ids_X_test.shape[0]])
data.append(['CICDDOS 2019', dos_X_train.shape[0], dos_X_test.shape[0]])
data.append(['IIOT', iiot_X_train.shape[0], iiot_X_test.shape[0]])
data = pd.DataFrame(data, columns=['Dataset Name', 'Training Size 80%', 'Testing Size 20%'])
data
```

Out[290]:

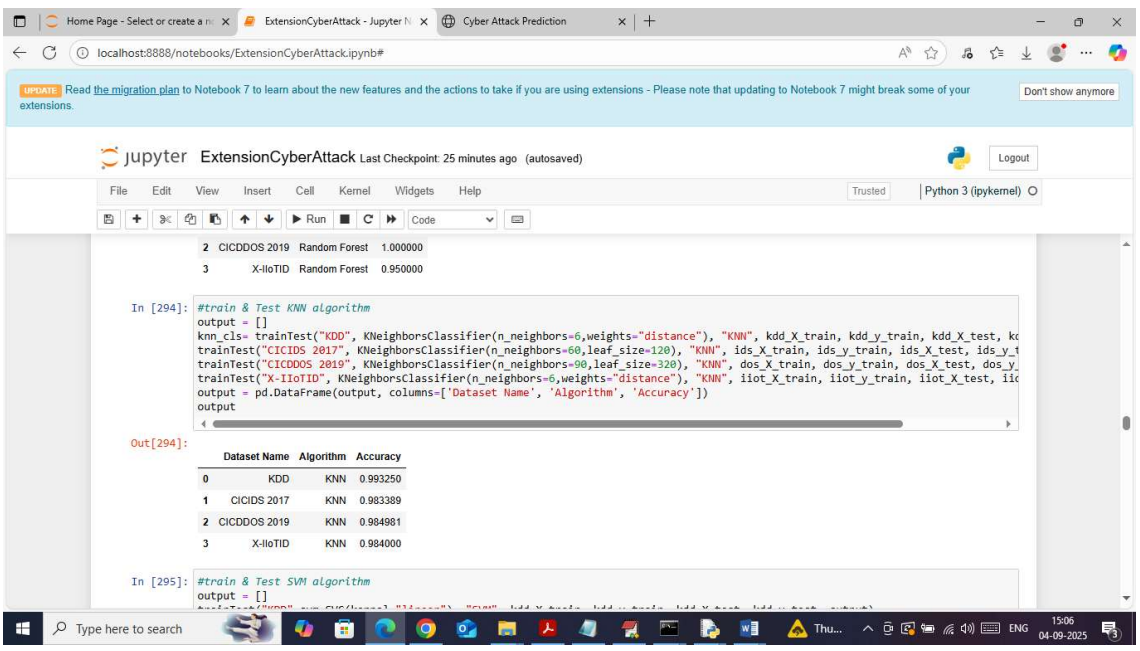
	Dataset Name	Training Size 80%	Testing Size 20%
0	NSL KDD	16000	4000
1	CICIDS 2017	3612	903
2	CICDDOS 2019	3194	799
3	IIOT	8000	2000

```
In [291]: #global variable to store all algorithms accuracy
accuracy = []
```

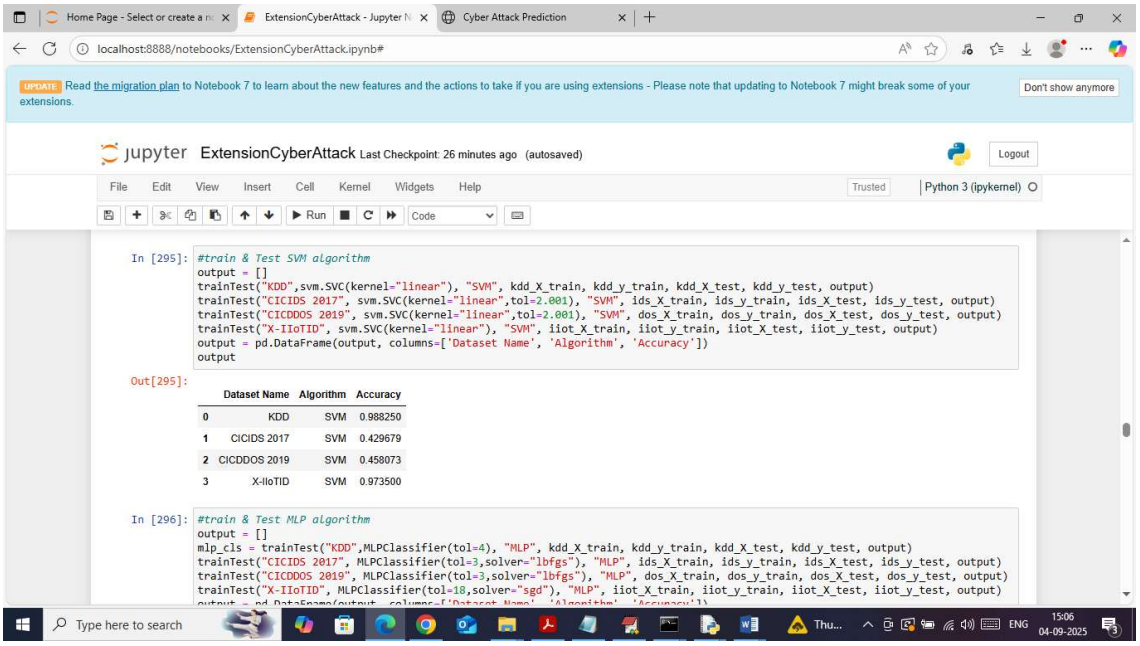
In above screen splitting all 4 datasets into train and test where application using 80% data for training and 20% for testing. In table format can see dataset names along with train and test size



In above screen training Random Forest on all 4 datasets and then performing prediction on 20% test data and then in table format can see Random Forest accuracy on each dataset



In above screen can see KNN accuracy on all 4 datasets



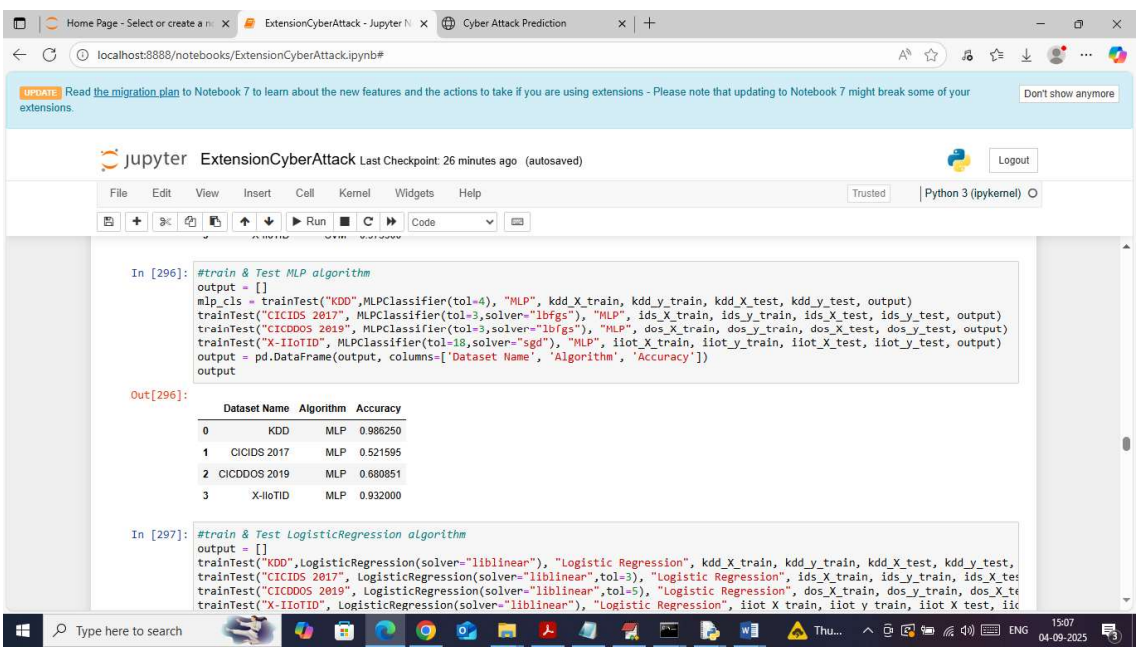
```
In [295]: #train & Test SVM algorithm
output = []
trainTest("KDD",svm.SVC(kernel="linear"), "SVM", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", svm.SVC(kernel="linear",tol=2.001), "SVM", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", svm.SVC(kernel="linear",tol=2.001), "SVM", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", svm.SVC(kernel="linear"), "SVM", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[295]:

	Dataset Name	Algorithm	Accuracy
0	KDD	SVM	0.988250
1	CICIDS 2017	SVM	0.429679
2	CICDDOS 2019	SVM	0.458073
3	X-IIoTID	SVM	0.973500

```
In [296]: #train & Test MLP algorithm
output = []
mlp_cls = trainTest("KDD",MLPClassifier(tol=4), "MLP", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", MLPClassifier(tol=3,solver="lbfgs"), "MLP", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", MLPClassifier(tol=3,solver="lbfgs"), "MLP", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", MLPClassifier(tol=18,solver="sgd"), "MLP", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
```

In above screen can see SVM performance on all 4 datasets



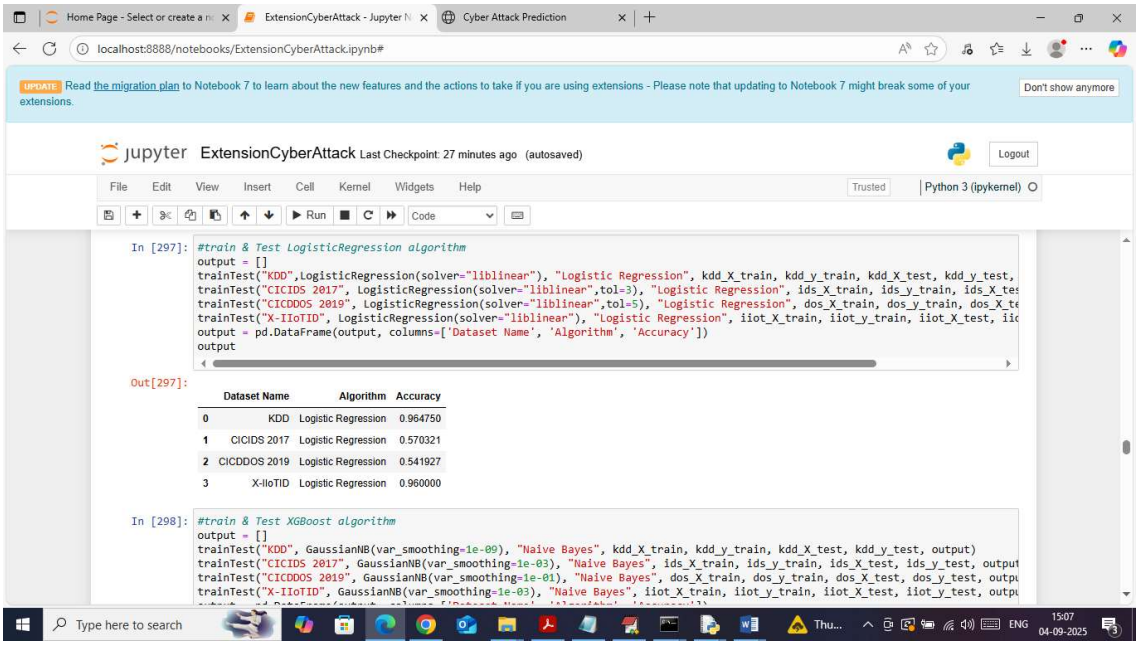
```
In [296]: #train & Test MLP algorithm
output = []
mlp_cls = trainTest("KDD",MLPClassifier(tol=4), "MLP", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", MLPClassifier(tol=3,solver="lbfgs"), "MLP", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output)
trainTest("CICDDOS 2019", MLPClassifier(tol=3,solver="lbfgs"), "MLP", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output)
trainTest("X-IIoTID", MLPClassifier(tol=18,solver="sgd"), "MLP", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output)
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[296]:

	Dataset Name	Algorithm	Accuracy
0	KDD	MLP	0.986250
1	CICIDS 2017	MLP	0.521595
2	CICDDOS 2019	MLP	0.680851
3	X-IIoTID	MLP	0.932000

```
In [297]: #train & Test LogisticRegression algorithm
output = []
trainTest("KDD",LogisticRegression(solver="liblinear"), "Logistic Regression", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test,
trainTest("CICIDS 2017", LogisticRegression(solver="liblinear",tol=3), "Logistic Regression", ids_X_train, ids_y_train, ids_X_te
trainTest("CICDDOS 2019", LogisticRegression(solver="liblinear",tol=5), "Logistic Regression", dos_X_train, dos_y_train, dos_X_te
trainTest("X-IIoTID", LogisticRegression(solver="liblinear"), "Logistic Regression", iiot_X_train, iiot_y_train, iiot_X_test, iic
```

In above screen can see MLP performance on all 4 datasets



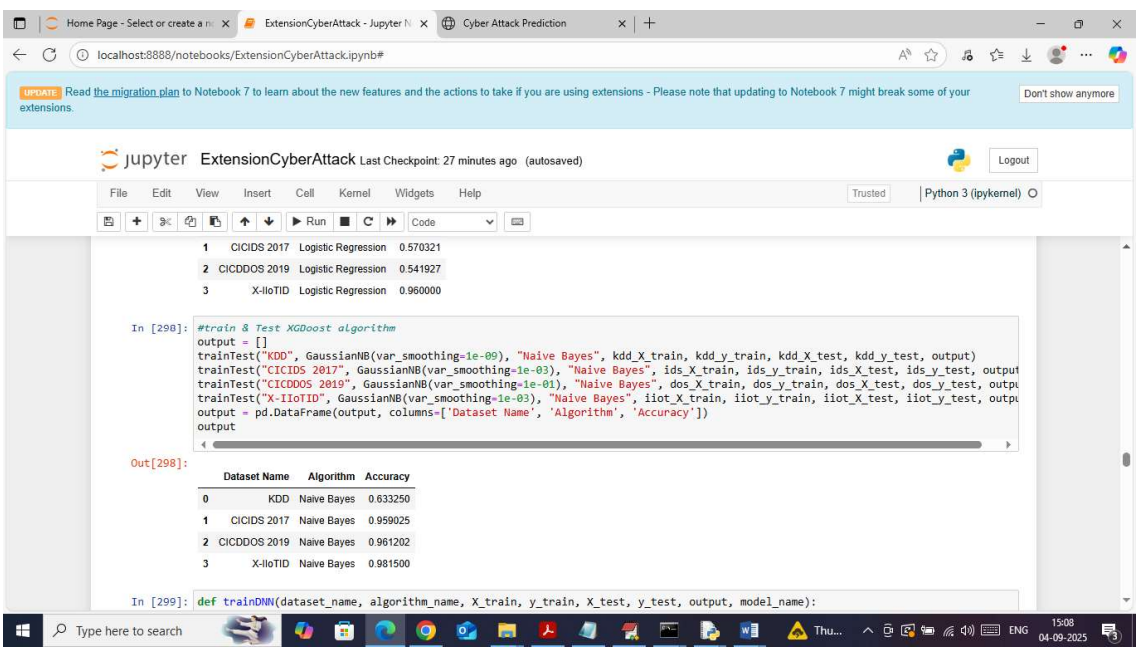
```
In [297]: #train & Test LogisticRegression algorithm
output = []
trainTest("KDD",LogisticRegression(solver="liblinear"), "Logistic Regression", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test,
trainTest("CICIDS 2017", LogisticRegression(solver="liblinear",tol=3), "Logistic Regression", ids_X_train, ids_y_train, ids_X_tes
trainTest("CICDDOS 2019", LogisticRegression(solver="liblinear",tol=5), "Logistic Regression", dos_X_train, dos_y_train, dos_X_te
trainTest("X-IIoTID", LogisticRegression(solver="liblinear"), "Logistic Regression", iiot_X_train, iiot_y_train, iiot_X_test, iio
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[297]:

	Dataset Name	Algorithm	Accuracy
0	KDD	Logistic Regression	0.964750
1	CICIDS 2017	Logistic Regression	0.570321
2	CICDDOS 2019	Logistic Regression	0.541927
3	X-IIoTID	Logistic Regression	0.960000

```
In [298]: #train & Test XGBoost algorithm
output = []
trainTest("KDD", GaussianNB(var_smoothing=1e-09), "Naive Bayes", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", GaussianNB(var_smoothing=1e-03), "Naive Bayes", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output
trainTest("CICDDOS 2019", GaussianNB(var_smoothing=1e-01), "Naive Bayes", dos_X_train, dos_y_train, dos_X_test, dos_y_test, outpu
trainTest("X-IIoTID", GaussianNB(var_smoothing=1e-03), "Naive Bayes", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, outpu
```

In above screen can see Logistic Regression performance on all 4 datasets



1	CICIDS 2017	Logistic Regression	0.570321
2	CICDDOS 2019	Logistic Regression	0.541927
3	X-IIoTID	Logistic Regression	0.960000

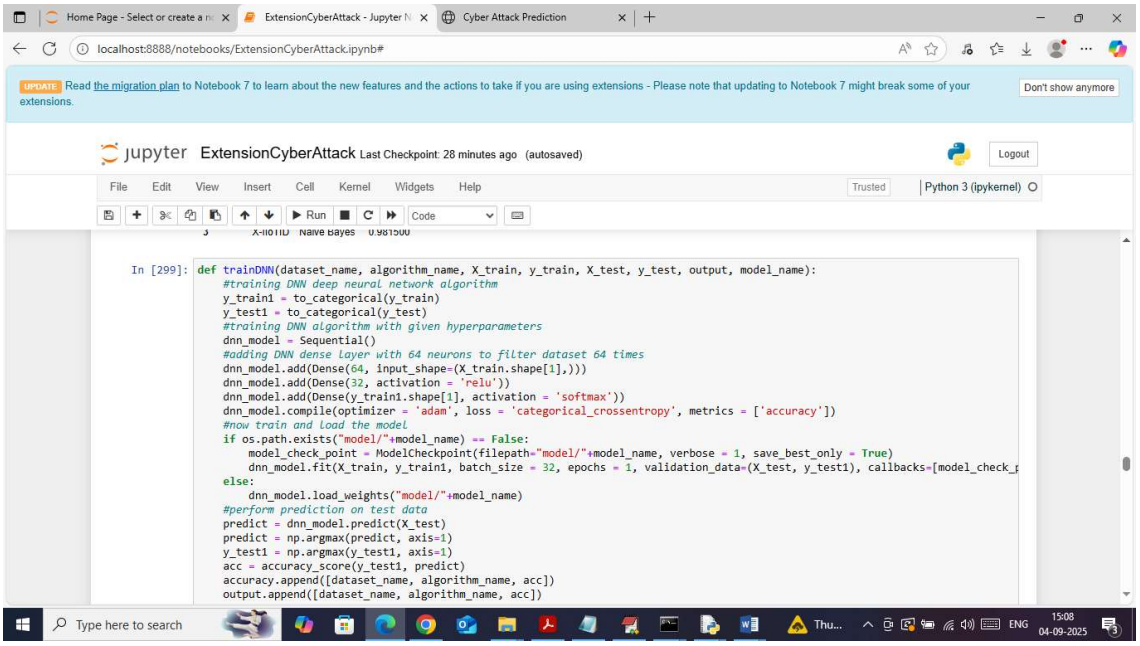
```
In [298]: #train & Test XGBoost algorithm
output = []
trainTest("KDD", GaussianNB(var_smoothing=1e-09), "Naive Bayes", kdd_X_train, kdd_y_train, kdd_X_test, kdd_y_test, output)
trainTest("CICIDS 2017", GaussianNB(var_smoothing=1e-03), "Naive Bayes", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output
trainTest("CICDDOS 2019", GaussianNB(var_smoothing=1e-01), "Naive Bayes", dos_X_train, dos_y_train, dos_X_test, dos_y_test, outpu
trainTest("X-IIoTID", GaussianNB(var_smoothing=1e-03), "Naive Bayes", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, outpu
output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
output
```

Out[298]:

	Dataset Name	Algorithm	Accuracy
0	KDD	Naive Bayes	0.633250
1	CICIDS 2017	Naive Bayes	0.959025
2	CICDDOS 2019	Naive Bayes	0.961202
3	X-IIoTID	Naive Bayes	0.981500

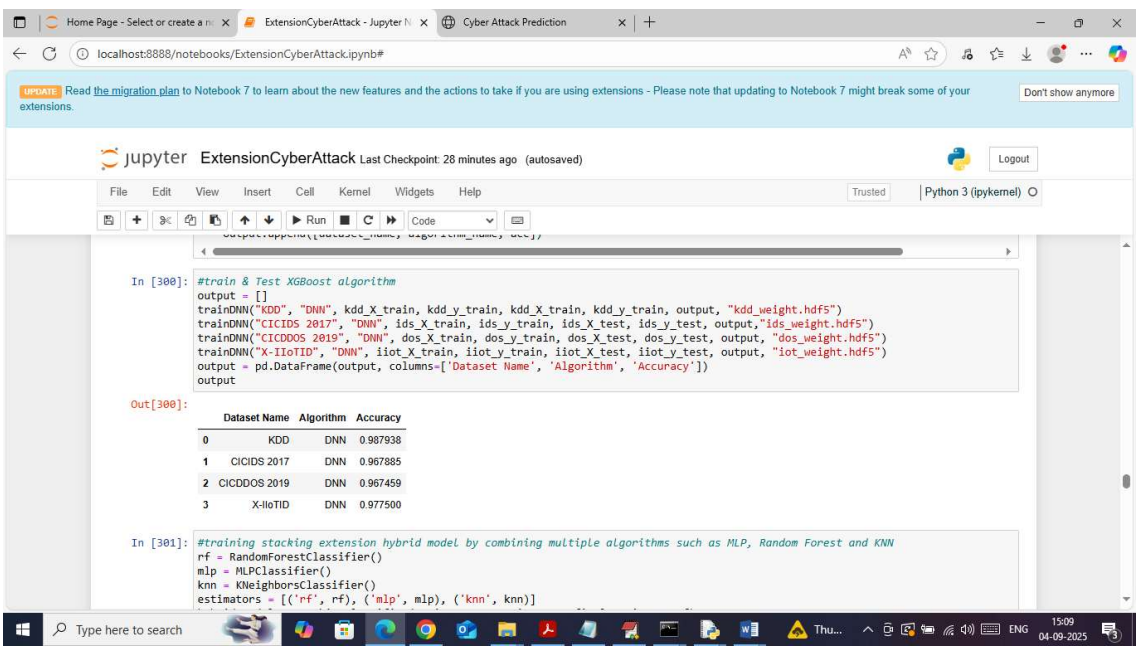
```
In [299]: def trainDNN(dataset_name, algorithm_name, X_train, y_train, X_test, y_test, output, model_name):
```

In above screen can see Naïve Bayes algorithm performance on all 4 datasets



```
In [299]: def trainDNN(dataset_name, algorithm_name, X_train, y_train, X_test, y_test, output, model_name):
              #training DNN deep neural network algorithm
              y_train1 = to_categorical(y_train)
              y_test1 = to_categorical(y_test)
              #training DNN algorithm with given hyperparameters
              dnn_model = Sequential()
              #adding DNN dense layer with 64 neurons to filter dataset 64 times
              dnn_model.add(Dense(64, input_shape=(X_train.shape[1],)))
              dnn_model.add(Dense(32, activation = 'relu'))
              dnn_model.add(Dense(y_train1.shape[1], activation = 'softmax'))
              dnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics = ['accuracy'])
              #now train and load the model
              if os.path.exists("model/"+model_name) == False:
                  model_check_point = ModelCheckpoint(filepath="model/"+model_name, verbose = 1, save_best_only = True)
                  dnn_model.fit(X_train, y_train1, batch_size = 32, epochs = 1, validation_data=(X_test, y_test1), callbacks=[model_check_p
              else:
                  dnn_model.load_weights("model/"+model_name)
              #perform prediction on test data
              predict = dnn_model.predict(X_test)
              predict = np.argmax(predict, axis=1)
              y_test1 = np.argmax(y_test1, axis=1)
              acc = accuracy_score(y_test1, predict)
              accuracy.append([dataset_name, algorithm_name, acc])
              output.append([dataset_name, algorithm_name, acc])
```

In above screen defining and training DNN algorithm on all 4 datasets and after executing above model will get below output



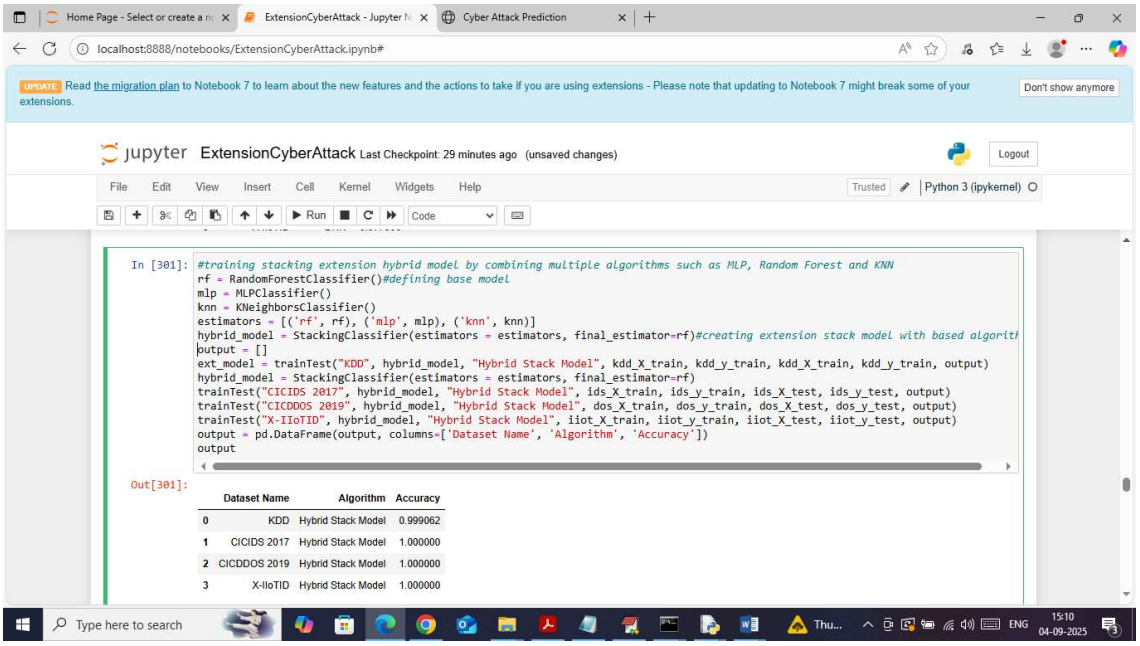
```
In [300]: #train & Test XGBoost algorithm
          output = []
          trainDNN("KDD", "DNN", kdd_X_train, kdd_y_train, kdd_X_train, kdd_y_train, output, "kdd_weight.hdf5")
          trainDNN("CICIDS 2017", "DNN", ids_X_train, ids_y_train, ids_X_test, ids_y_test, output,"ids_weight.hdf5")
          trainDNN("CICDDOS 2019", "DNN", dos_X_train, dos_y_train, dos_X_test, dos_y_test, output, "dos_weight.hdf5")
          trainDNN("X-IIoTID", "DNN", iiot_X_train, iiot_y_train, iiot_X_test, iiot_y_test, output, "iot_weight.hdf5")
          output = pd.DataFrame(output, columns=['Dataset Name', 'Algorithm', 'Accuracy'])
          output
```

Out[300]:

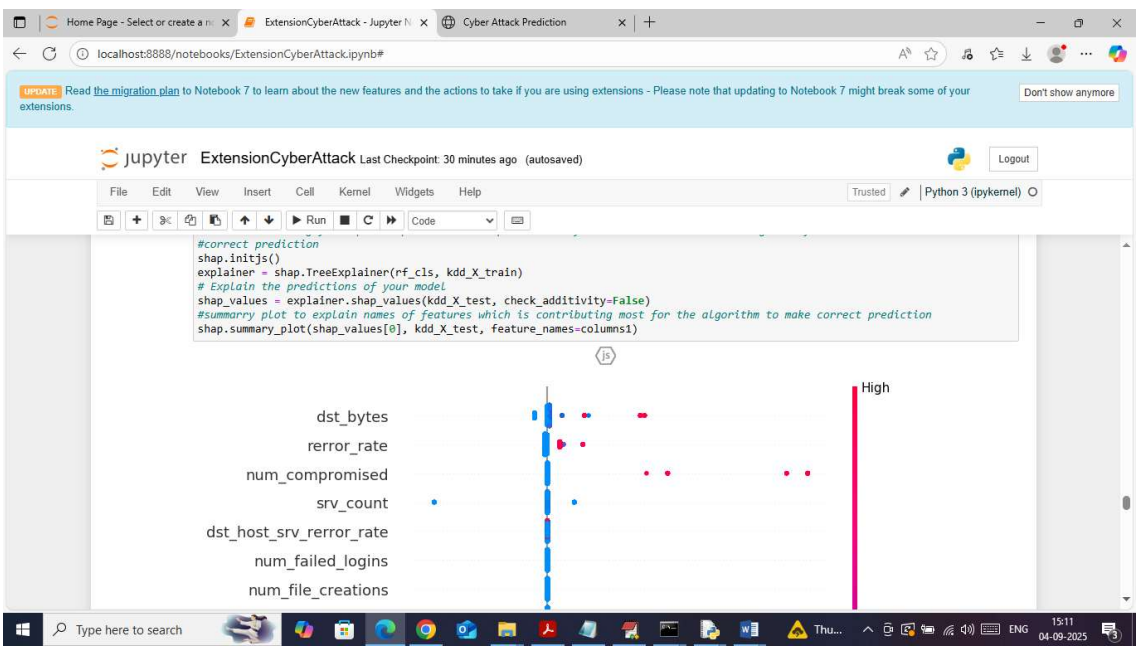
	Dataset Name	Algorithm	Accuracy
0	KDD	DNN	0.987938
1	CICIDS 2017	DNN	0.967885
2	CICDDOS 2019	DNN	0.967459
3	X-IIoTID	DNN	0.977500

```
In [301]: #training stacking extension hybrid model by combining multiple algorithms such as MLP, Random Forest and KNN
          rf = RandomForestClassifier()
          mlp = MLPClassifier()
          knn = KNeighborsClassifier()
          estimators = [('rf', rf), ('mlp', mlp), ('knn', knn)]
```

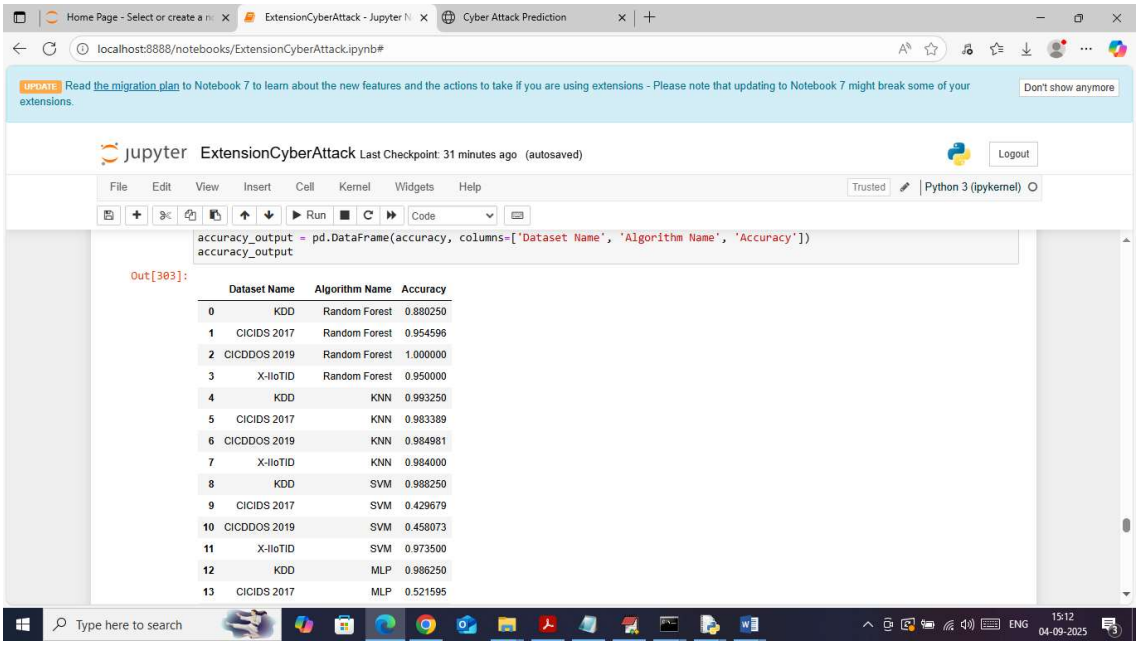
In above screen can see DNN accuracy on all 4 datasets



In above screen training extension hybrid algorithms and after training extension model got 99 and 100% accuracy on all 4 datasets

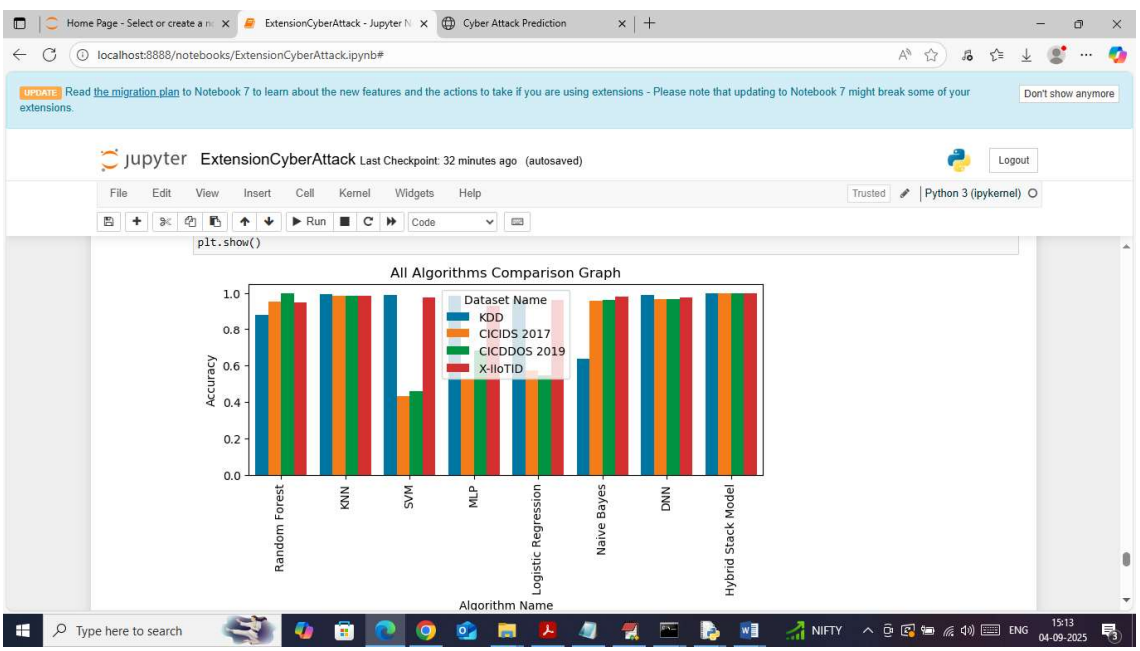


In above screen defining SHAP explanation tool on trained and test data and then SHAP explaining which features contributing most for correct class label prediction, in above graph feature names on left side with most number of dots will be consider as high relevant features

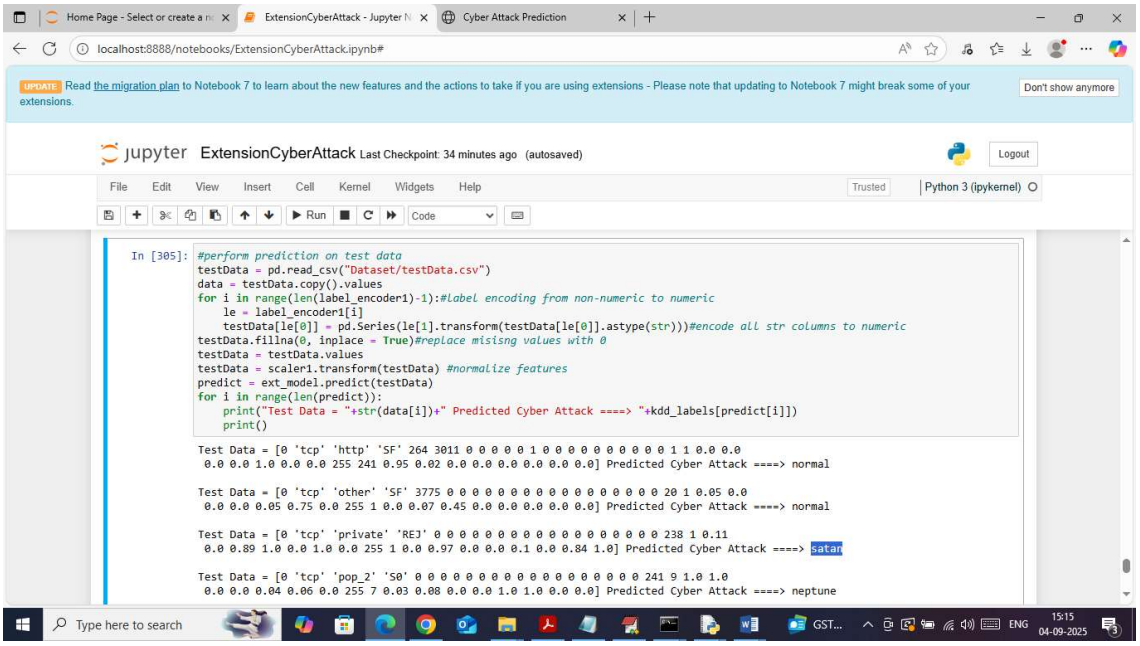


	Dataset Name	Algorithm Name	Accuracy
0	KDD	Random Forest	0.880250
1	CICIDS 2017	Random Forest	0.954596
2	CICDDOS 2019	Random Forest	1.000000
3	X-IIoTID	Random Forest	0.950000
4	KDD	KNN	0.993250
5	CICIDS 2017	KNN	0.983389
6	CICDDOS 2019	KNN	0.984981
7	X-IIoTID	KNN	0.984000
8	KDD	SVM	0.988250
9	CICIDS 2017	SVM	0.429679
10	CICDDOS 2019	SVM	0.458073
11	X-IIoTID	SVM	0.973500
12	KDD	MLP	0.986250
13	CICIDS 2017	MLP	0.521595

In above screen in table format showing all algorithms accuracy on each dataset



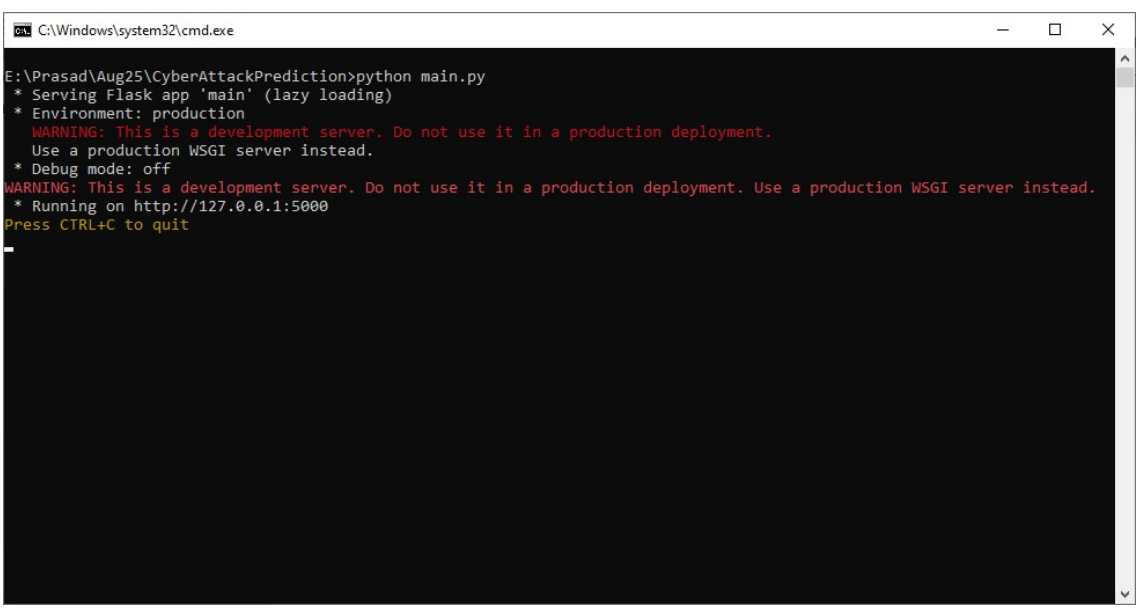
In above screen showing comparison graph between all algorithms with all datasets. In above graph x-axis represents 'algorithm names' and y-axis represents accuracy and different colour bars represents different dataset names. In above graph can see Extension Hybrid model got high accuracy



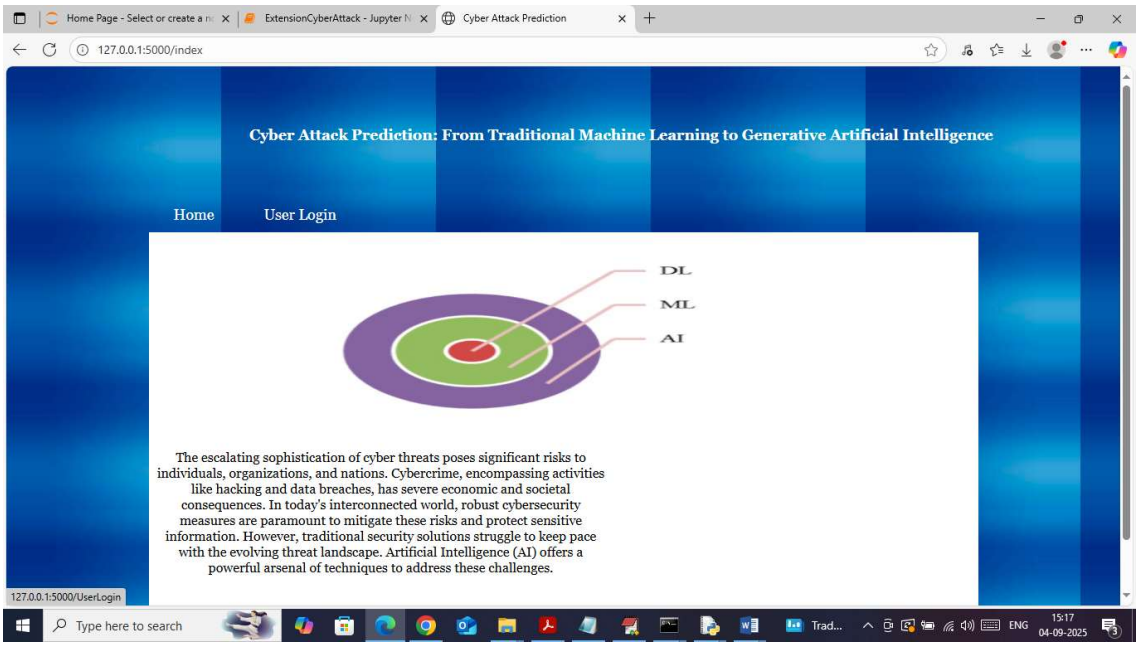
In above screen loading and processing test data values and then applying extension model to predict attack type. In above output area in square bracket can see test data values and after =➔ symbol can see predicted attack type

WEB OUTPUT

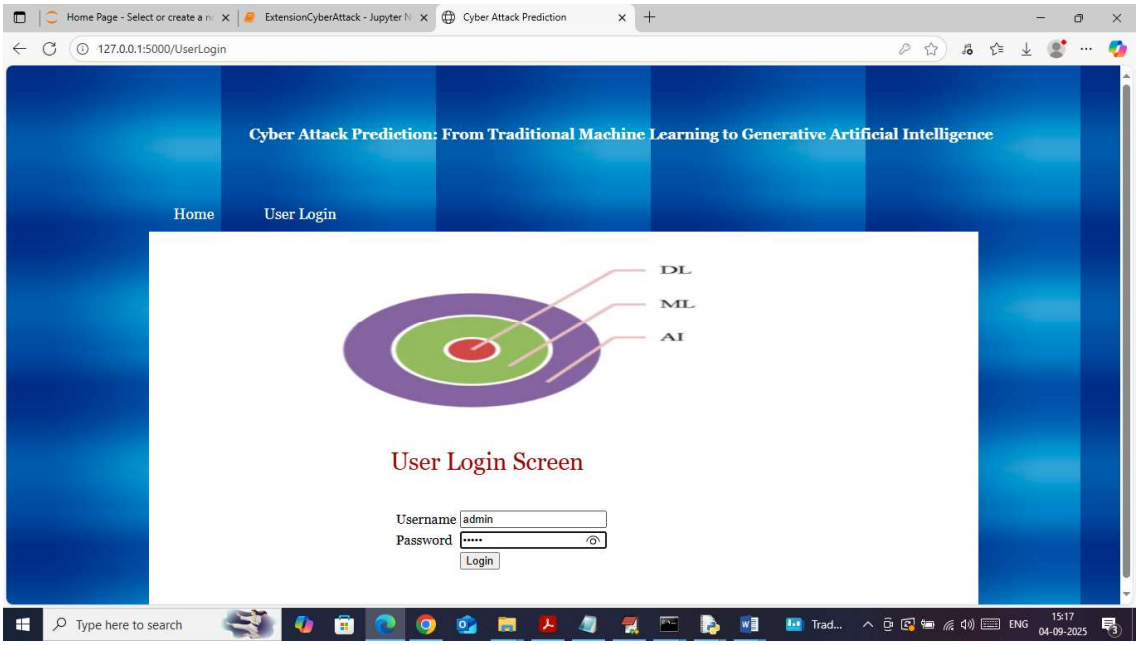
To run web prediction double click on 'runFlask.bat' file to start flask server and then will get below page



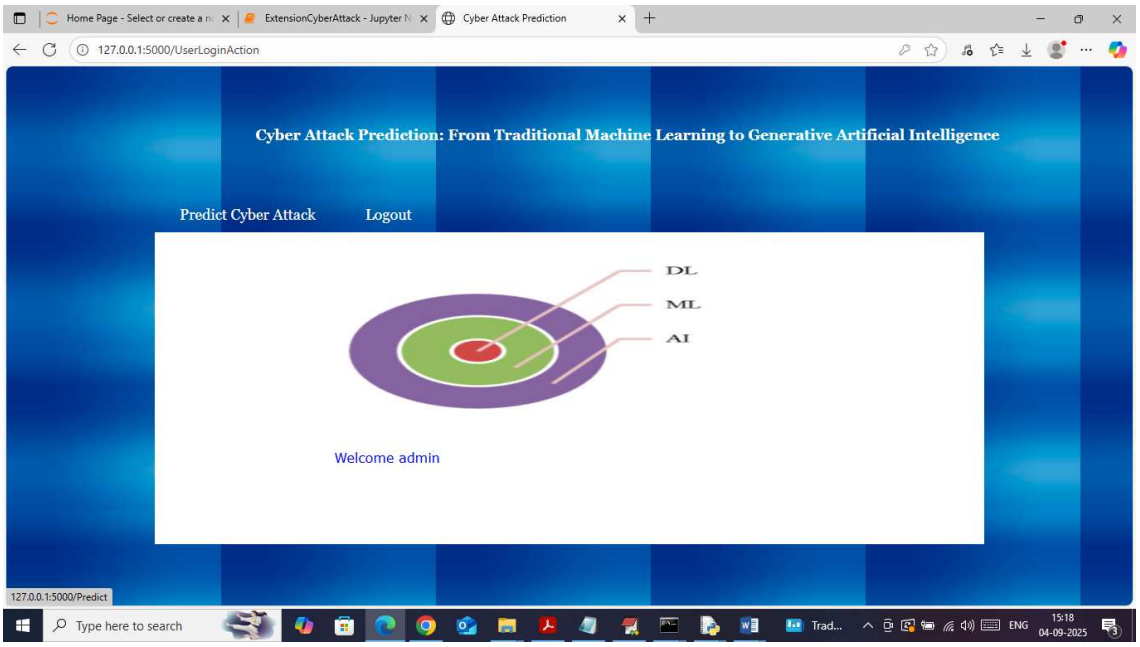
In above screen flask server started and now open browser and enter URL as http://127.0.0.1:5000/index and then press enter key to get below page



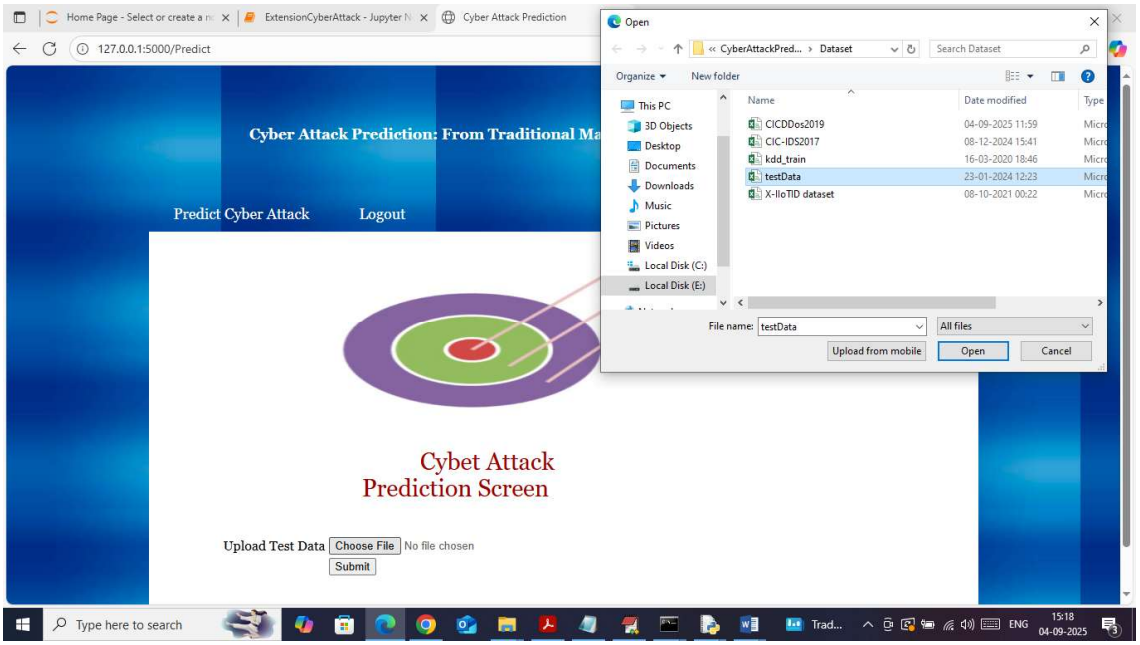
In above screen click on 'User Login' link to get below page



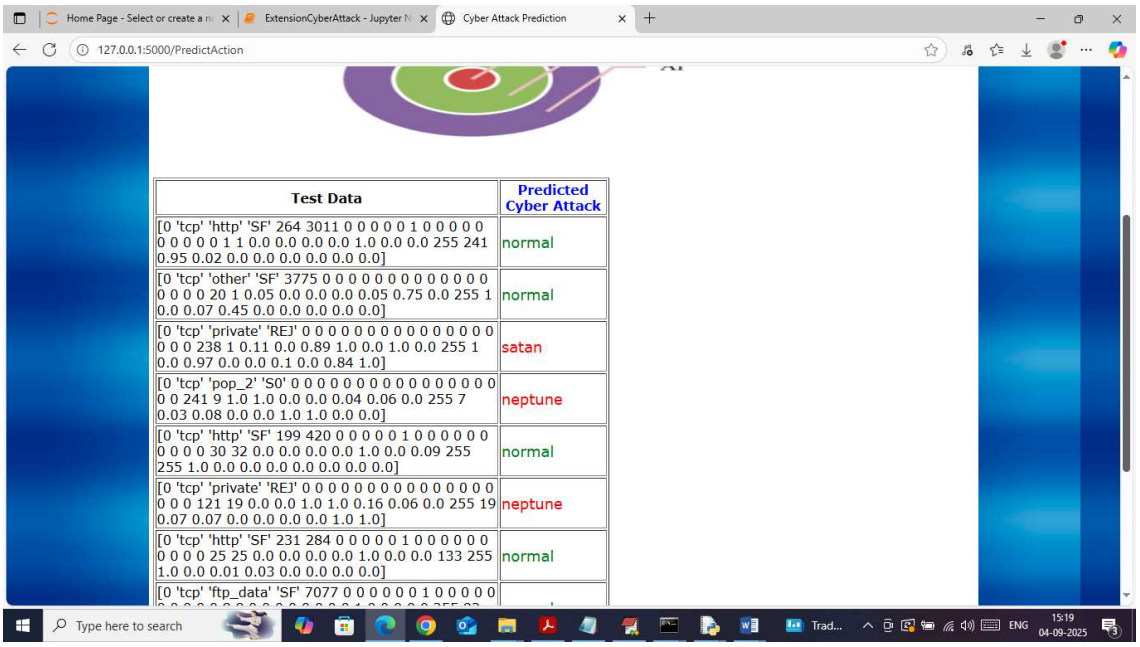
In above screen user is login by entering username and password as 'admin and admin'. After login will get below page



In above screen click on ‘Predict Cyber Attack’ link to get below page



In above screen selecting and uploading test data file and then click on buttons to get below page



In above screen in first column can see test data values and in second column can see predicted attack type.